**Milestone Two Software Design and Engineering Narrative**

The artifact I selected is an interactive three-dimensional desktop scene that I originally created in CS 330 Computational Graphics and Visualization. The project was developed in C++ with OpenGL, GLFW, GLEW, and GLM. It recreates a desktop environment from primitive meshes, including a table surface, a laptop, a transparent drink cup, a candle, and a succulent. The original artifact uses transformations to position and scale each object, texture mapping and materials to define surface appearance, multiple light sources to illuminate the scene, and keyboard and mouse controls to navigate the camera. I created the original version during CS 330 and enhanced it in 2026 for the software design and engineering category of my CS 499 ePortfolio.

This milestone focuses on the quality of the software that prepares and renders the scene. I preserved the original code as evidence of where the project began and created a separate enhanced version so a reviewer can compare the two implementations. The submission package includes both versions, their shaders, textures, Visual Studio project files, and the dependencies needed by the enhanced project. Generated executables, object files, caches, and debugging databases are excluded because they are reproducible build products rather than source artifacts.

I selected this artifact because it demonstrates more than the ability to draw a collection of shapes. The scene coordinates several software components that must work together: the application entry point establishes the graphics context, the view manager controls camera and projection behavior, the scene manager owns scene resources and issues drawing commands, the shader manager sends data to the GPU, and the vertex and fragment shaders calculate the final image. This combination makes the artifact useful for demonstrating C++ development, object-oriented organization, graphics programming, resource management, debugging, and the ability

to reason about behavior across CPU and GPU code. It also supports my software engineering goals because the same practices used to improve this graphics application, which include clear responsibilities, validation, predictable lifecycle management, and useful failure behavior, apply to larger applications.

The original scene successfully modeled recognizable objects and supported camera navigation, but the review identified weaknesses that reduced its reliability and maintainability. The texture counter was not explicitly initialized, texture storage could be written without a capacity check, duplicate tags could be registered, and an invalid tag could send a negative sampler slot to the shader. The cleanup function called glGenTextures instead of glDeleteTextures, which created identifiers rather than releasing GPU resources. Repeated texture-loading statements made the asset list harder to review, and repeated calls to PrepareScene could duplicate scene resources. Startup code also continued without fully checking GLFW, window, and shader initialization. In addition, incorrect material assignments and an incomplete lighting equation contributed to inconsistent and overly dark rendering.

I improved the artifact by explicitly initializing the scene manager's dependencies and state, making scene preparation idempotent, and centralizing texture definitions in a small data-driven asset list. Texture creation now rejects duplicate tags, checks the fixed registry capacity before writing, frees temporary image and OpenGL resources on unsupported formats, and uses mipmapped filtering. Texture lookup now fails safely by disabling texture use and reporting the missing tag instead of sending an invalid index. Cleanup now collects the active texture identifiers and releases them with glDeleteTextures from the scene manager destructor while the graphics context is active. The material collection is cleared before it is rebuilt, and the grass, stainless steel, and onyx records are assigned to their correct objects. String lookup parameters

are passed by const reference to communicate that the methods do not modify them and to avoid unnecessary copies.

I also strengthened the application boundary. The program now checks GLFW initialization, display-window creation, and shader-program creation and returns a failure code after orderly cleanup when a required service is unavailable. The window title and file headers identify the artifact and my CS 499 enhancement work. In the rendering pipeline, I corrected the fragment shader so diffuse and specular light colors affect the Phong result, added a controlled global ambient term, adjusted the material values, replaced missing texture filenames with assets included in the project, and increased the overhead and ambient illumination. These changes preserve the warm candle and cool side light while making shadowed objects readable. The result is a clearer scene supported by code that is safer to maintain and easier for another developer to inspect.

This enhancement made the strongest progress toward the fourth course outcome: demonstrating well-founded techniques, skills, and tools to implement computing solutions that deliver value. The enhanced application applies C++ object-oriented design, OpenGL resource lifecycle management, defensive validation, shader programming, structured error handling, and Visual Studio project organization to improve a working graphics solution. The value is observable in two ways. Users receive a more visible and consistent scene, while developers receive clearer initialization, safer cleanup, centralized asset configuration, and actionable error messages. These changes show that software quality includes reliability and maintainability in addition to visible functionality.

The work also supports the second outcome involving professional-quality communication. The revised file headers, focused comments, meaningful diagnostic messages,

enhancement summary, organized before-and-after package, and this narrative provide technical information for an instructor, peer developer, or employer. They explain the purpose and consequences of the changes without requiring the reader to infer every decision from a raw code diff. The fifth outcome, developing a security mindset, is partially addressed through defensive behavior. Capacity validation prevents an out-of-bounds write to the texture array, failed lookups no longer propagate invalid values, duplicate asset identifiers are rejected, initialization failures stop the program safely, and temporary resources are released along error paths. Although this is not a networked application, these controls reduce undefined behavior and show the practice of treating external files and runtime state as values that must be validated.

My Module One plan anticipated a broader object-oriented scene architecture. During implementation, I narrowed this milestone to a practical foundation: correct resource ownership, data-driven asset setup, defensive startup behavior, material and shader corrections, and improved documentation. This still represents substantial progress in software engineering and design, but I am updating my outcome-coverage plan so I do not claim that the first enhancement completes the algorithmic or database outcomes. The third outcome will be addressed more directly in the algorithms and data structures milestone through structured scene-object storage, selection, geometric intersection tests, and evaluation of lookup trade-offs. Database persistence and parameterized operations will be addressed in the database milestone. The first outcome, collaboration for diverse audiences, remains partial because the artifact is individual work; I will strengthen it by incorporating instructor feedback, documenting design decisions, and presenting the finished portfolio so technical and nontechnical reviewers can make informed decisions about the project.

The most important lesson from this enhancement was that a visible graphics problem can originate in several layers. Increasing a light's numeric intensity did not fully solve the dark scene because the fragment shader was not consistently applying each light's diffuse and specular colors, some material definitions referenced the wrong variables, and several texture filenames did not match the assets in the project. I had to trace information from SceneManager through shader uniforms and into the fragment-lighting calculation rather than treating brightness as a single setting. That process improved my understanding of how application code, data, GPU state, and shader logic form one rendering pipeline. It also reinforced the value of changing one concern at a time and checking whether the result supports the intended visual hierarchy.

A second challenge was improving the framework without unnecessarily rewriting it. The original project already separated scene, view, shader, and mesh responsibilities, so replacing the entire architecture would have increased risk and made the before-and-after comparison harder to evaluate. I focused on defects with concrete consequences and refactored repeated asset setup into a data-driven loop. This approach taught me to distinguish between a large redesign and an incremental engineering improvement. A smaller change can demonstrate mature design judgment when it fixes resource leaks, prevents invalid state, improves diagnostics, and creates a safer foundation for later features.

I also learned that cleanup code deserves the same review as rendering code. The original DestroyGLTextures method looked reasonable at a glance, but it generated new texture identifiers instead of deleting the existing ones. Correcting that defect requires thinking about ownership and destruction order: the scene manager must release GPU resources before the window and its OpenGL context are terminated. I applied the same lifecycle reasoning to early startup failures by adding cleanup before returning. This was a useful reminder that software

engineering includes what happens when an application cannot continue, not only what happens during a successful frame.

The primary remaining limitation is environment-specific validation. The supplied solution and native graphics libraries target Visual Studio on Windows, so the final runtime build should be completed in that environment after submission. I performed source-level checks, verified that referenced texture assets and material tags exist, preserved a complete comparison copy, and tested the final ZIP for structural integrity. Before publishing the artifact to my ePortfolio, I will also incorporate instructor feedback, complete a clean Visual Studio build, exercise camera and projection controls, observe the scene from multiple angles, and confirm that application shutdown releases resources without errors. Overall, the enhancement moved the project from a visually functional course assignment toward a more dependable and professionally presented software artifact.